

Advances in Lock-Free Programming

Mainack Mondal
Sandip Chakraborty

CS 60203
Autumn 2026



What we have seen so far ...

- Locks in OS
 - What are locks and why are they required?
 - Different types of Locks
 - Why are Locks Bad?
- Lock-Free Programming
 - Definition, Different Lock Free Primitives
 - Examples of lockless Data Structures
 - Advantages
 - Problems ~ ABA Problem

Outline

- Lock-Free Primitives
 - Hardware
 - Software
- Read-Copy-Update(RCU) in Linux
- Lock-Free APIs in Programming Languages
 - Java
 - C/C++
- Progress Guarantees
- Memory Ordering
 - Store buffers, memory_order, hardware differences
- Safe Memory Reclamation
 - Hazard Pointers, Epoch-Based Reclamation

Lock-Free Primitives

Lock-Free Primitives in Hardware

Ever wondered... How CAS instruction is implemented in hardware ?

- Using a special compare and exchange instruction (x86)
 - **CMPXCHG**

Instruction Format:

CMPXCHG **reg/mem32**, **reg32**

destination
register/memory

source register

same/similar
instructions exists in
other architectures.
[Read more](#)

How does it work ?

The CMPXCHG instruction

```
accumulator = %eax
TEMP = DEST

IF accumulator = TEMP
    THEN
        ZF := 1;
        DEST := SRC;
    ELSE
        ZF := 0;
        accumulator := TEMP;
        DEST := TEMP;
FI;
```

ZF : Zero Flag (also known as EFLAG) is a status flag in the FLAGS register. [Read more](#)

Similar instructions exist for 8 bit, 16 bit and 64 bit architectures. [Read more](#)

The CMPXCHG instruction

- The CMPXCHG instruction is not completely atomic !!
- It is atomic, but only on single core, not for multiple cores
- To make it atomic on multiple cores a special prefix is used

LOCK CMPXCHG reg/mem32, reg32

↑
prefix

Note: Don't confuse it with the lock that you have studied. It is just a part of the instruction to make it atomic across multiple cores

Lock-Free Primitives in Software

- In software, such primitives are provided by compilers
- Underlying these functions use hardware primitives
- For eg: GCC provides some atomic builtins :
 - `__atomic_compare_exchange`
 - `__atomic_fetch_add`
 - `__atomic_test_and_set`
 - `__atomic_is_lock_free`... and many more

Read more about them here:

https://gcc.gnu.org/onlinedocs/gcc-4.8.2/gcc/_005f_005fatomic-Builtins.html

We will see more such standard APIs later

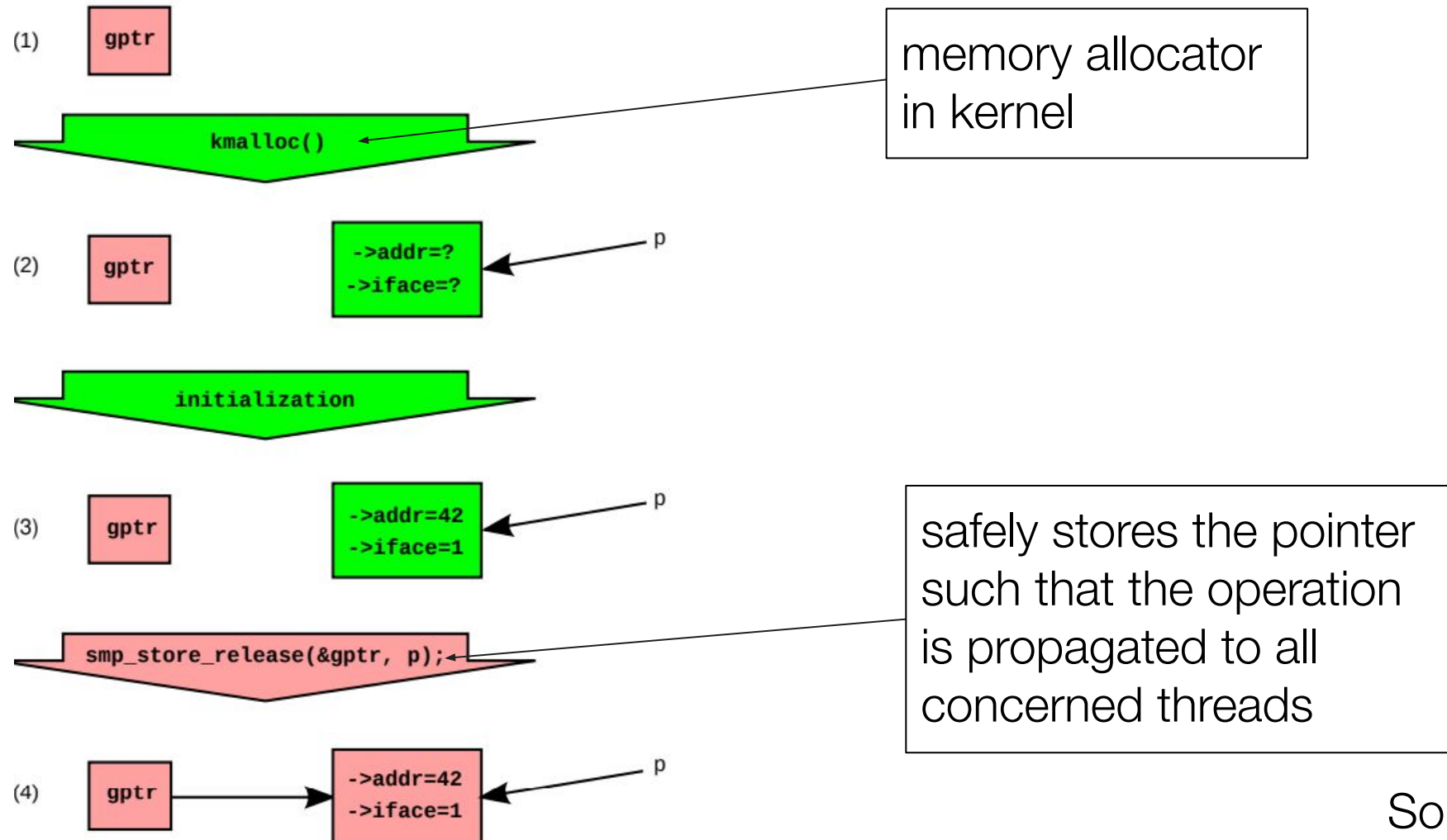
Read-Copy-Update(RCU) in Linux

Read-Copy-Update(RCU) in Linux

- Another synchronization mechanism, added to linux kernel in 2002
- Supports concurrency between multiple reader and a single updater
- no over-head from read-side primitive
- considered as one of the safest data-structures
- uses cache-line and memory very efficiently
- provides lock-free read critical section
- locks when a writer is compatible with readers
- **preemption** is not allowed in the read critical section

RCU : Motivation

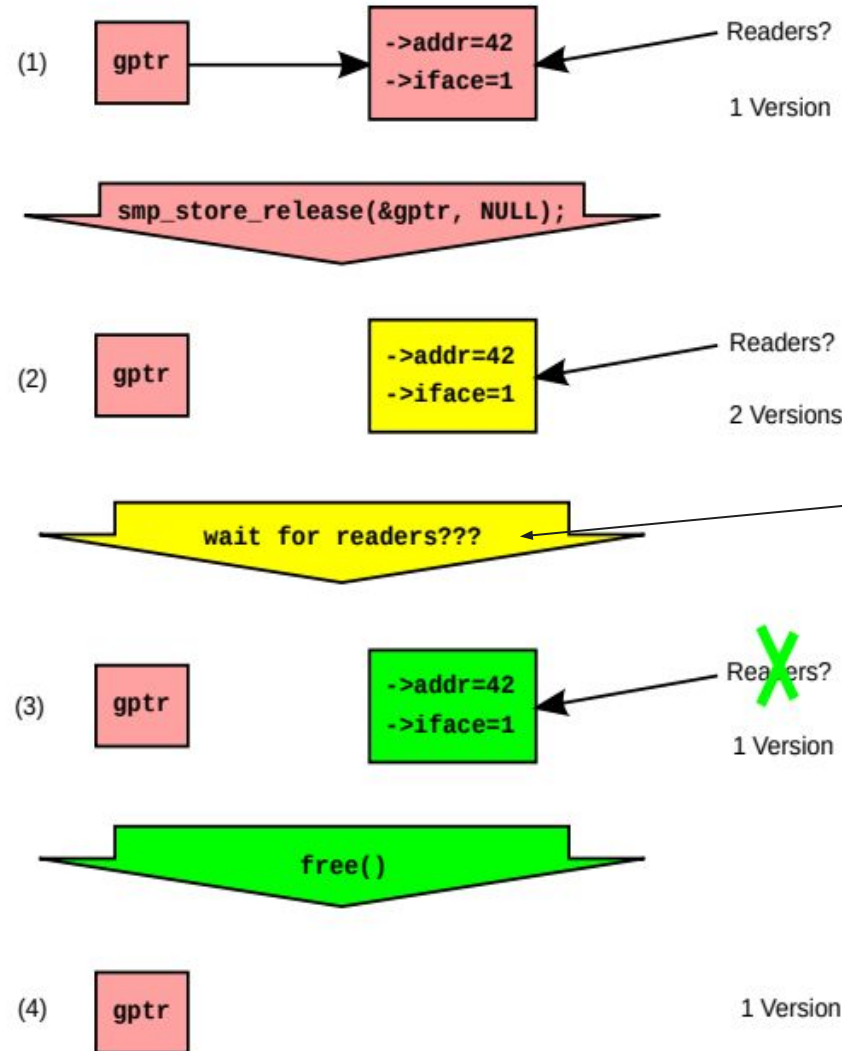
Lets see an example of simple pointer update



Source: [Link](#)

RCU : Motivation

Freeing the pointer



How do you do that ?
read sec 9.5.1.3 : [Link](#)

RCU Core APIs

	Primitive	Purpose
<i>Readers</i>	<code>rcu_read_lock()</code>	Start an RCU read-side critical section.
	<code>rcu_read_unlock()</code>	End an RCU read-side critical section.
	<code>rcu_dereference()</code>	Safely load an RCU-protected pointer.
<i>Updaters</i>	<code>synchronize_rcu()</code>	Wait for all pre-existing RCU read-side critical sections to complete.
	<code>call_rcu()</code>	Invoke the specified function after all pre-existing RCU read-side critical sections complete.
	<code>rcu_assign_pointer()</code>	Safely update an RCU-protected pointer.

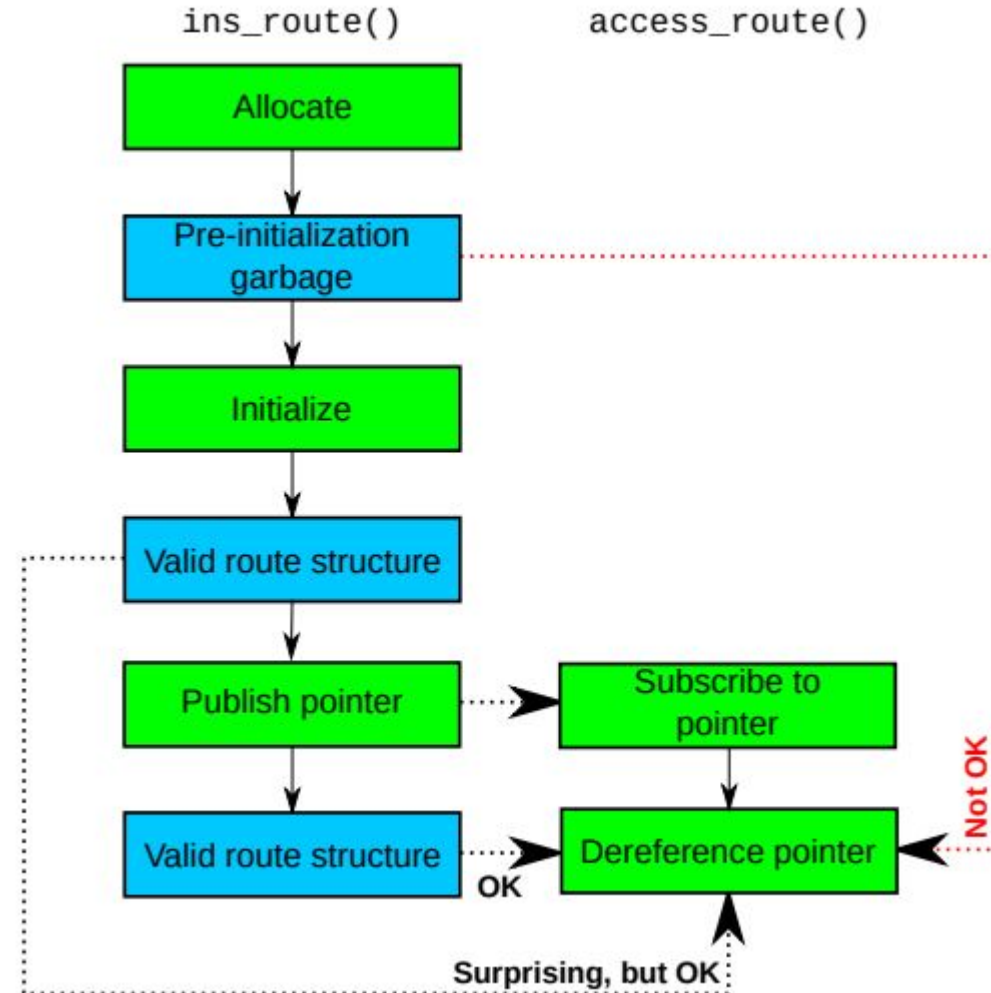
Quick Question: Can you use RCU in user space ?

Properties of RCU

- Reads need not wait for updates
 - provides low-cost/no-cost readers leading to low overhead and great scalability
 - allows RCU readers and updaters to make useful concurrent forward progress.
- Each reader has a coherent view of each object
 - Ensured by:
 - maintaining multiple versions of objects
 - using update-side primitives like `synchronize_rcu()` to ensure objects are not freed until all readers have completed

RCU Fundamentals

- Publish-Subscribe Mechanism
 - Readers are subscribers → subscribing to the current version of the RCU-protected data item
 - Updaters are publishers



RCU Fundamentals

- Wait For Pre-Existing RCU Readers

“The great advantage of RCU is that it can wait for each of (say) 20,000 different things without having to explicitly track each and every one of them, and without having to worry about the performance degradation, scalability limitations, complex deadlock scenarios, and memory-leak hazards that are inherent in schemes using explicit tracking”

- Maintain Multiple Versions of Recently Updated Objects

Outline

- Lock-Free Primitives
 - Hardware
 - Software
- Read-Copy-Update(RCU) in Linux
- Lock-Free APIs in Programming Languages
 - Java
 - C/C++
- Progress Guarantees
- Memory Ordering
 - Store buffers, memory_order, hardware differences
- Safe Memory Reclamation
 - Hazard Pointers, Epoch-Based Reclamation

Lock-Free APIs in Programming Languages

Lock-Free APIs in Java

- numerous packages consisting of lock-free primitives
- one such is `java.util.concurrent.atomic`
 - provides various features like atomic integers, booleans, references etc.
 - Examples:

i) `AtomicInteger`

Provides atomic operations on an integer, such as `get()`, `set()`, `incrementAndGet()`, and `compareAndSet()`

```
AtomicInteger atomicInt = new AtomicInteger(0);  
int currentValue = atomicInt.get();  
atomicInt.incrementAndGet();  
boolean success = atomicInt.compareAndSet(0, 1);
```

Lock-Free APIs in Java (contd.)

ii) AtomicReference

Provides atomic operations on objects or references to objects. This is useful for lock-free linked data structures

```
AtomicReference<String> atomicRef = new AtomicReference<>("initial");  
boolean success = atomicRef.compareAndSet("initial", "updated");
```

Similarly, there are multiple functionalities like AtomicBoolean, AtomicLong etc. [Read more](#)

Lock-Free APIs in C/C++

- in C++ we have the `atomic` library for lock-free programming
- `std::atomic` provides various atomic data types like:
 - `atomic_int`
 - `atomic_bool`
 - `atomic_char`
 - `atomic_intptr_t`
 - ...
- provides templated access to atomic primitives
 - i.e. `std::atomic<T>`
 - can use with user defined data types (UDT) !!

Lock-Free APIs in C/C++ (contd.)

Member functions

(constructor)	constructs an atomic object (public member function)
operator=	stores a value into an atomic object (public member function)
is_lock_free	checks if the atomic object is lock-free (public member function)
store	atomically replaces the value of the atomic object with a non-atomic argument (public member function)
load	atomically obtains the value of the atomic object (public member function)
operator T	loads a value from an atomic object (public member function)
exchange	atomically replaces the value of the atomic object and obtains the value held previously (public member function)
compare_exchange_weak compare_exchange_strong	atomically compares the value of the atomic object with non-atomic argument and performs atomic exchange if equal or atomic load if not (public member function)
wait (C++20)	blocks the thread until notified and the atomic value changes (public member function)
notify_one (C++20)	notifies at least one thread waiting on the atomic object (public member function)
notify_all (C++20)	notifies all threads blocked waiting on the atomic object (public member function)

What ??

Lock-Free APIs in C/C++ (contd.)

Specialized member functions

Specialized for integral, floating-point(since C++20) and pointer types

<code>fetch_add</code>	atomically adds the argument to the value stored in the atomic object and obtains the value held previously (public member function)
<code>fetch_sub</code>	atomically subtracts the argument from the value stored in the atomic object and obtains the value held previously (public member function)
<code>operator+=</code> <code>operator-=</code>	adds to or subtracts from the atomic value (public member function)

Specialized for integral and pointer types only

<code>fetch_max</code> (C++26)	atomically performs <code>std::max</code> between the argument and the value of the atomic object and obtains the value held previously (public member function)
<code>fetch_min</code> (C++26)	atomically performs <code>std::min</code> between the argument and the value of the atomic object and obtains the value held previously (public member function)

Lock-Free APIs in C/C++ : Example

```
#include <atomic>
std::atomic<int> counter(0); // Atomic shared counter
void increment() {
    for (int i = 0; i<1000; i++)
        ++counter; // Atomic increment (lock-free)
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 100; ++i) {
        threads.push_back(std::thread(increment));
    }
    for (auto& t : threads) {
        t.join();
    }
    std::cout << "Final counter value: " << counter << std::endl;
    return 0;
}
```

Progress Guarantees: How “Lock-Free” Are We?

- “Lock-free” is actually a family of guarantees, not one thing:
 - Obstruction-free: a thread makes progress if it runs alone (no other thread interferes)
 - Lock-free: the system as a whole always makes progress — some thread completes in a bounded number of steps, even if individual threads can be starved
 - Wait-free: every thread makes progress in a bounded number of its own steps, no matter what others do
- Our CAS-retry-loop stack (push/pop with a while(!CAS) loop) is lock-free, not wait-free
 - A thread can retry forever if it keeps losing the CAS race to others — rare in practice, but possible
- Wait-free algorithms exist but are complex and often slower in the common case — mostly used where hard real-time bounds matter (e.g. audio callbacks, some HFT order-entry paths)

Memory Ordering

Wait, Rewind — One More Outcome

- Recall the multicore-class litmus test:

■ thread 1: store 1→y ; load x thread 2: store 1→x ;
 load y

- We listed 6 outcomes from interleaving the instructions — all had $x=1$ or $y=1$
- On real x86 hardware, a 7th outcome is observed: $(x=0, y=0)$
- No interleaving of these four instructions can produce that! Something below the instruction level is reordering things.

Why: Store Buffers

- Each core has a small store buffer between it and the cache
 - A store is written to the local store buffer immediately — the core moves on — and drains to cache/other cores slightly later
 - A core's own later load can be served from its own store buffer (store-to-load forwarding) before that store is globally visible
- So: both cores can have their store still sitting in their private buffer when they issue their load — both loads see the old value → (x=0, y=0)
- This exact pattern (StoreLoad reordering) is the only reordering x86 allows — which is why we see this outcome and not others

memory_order: Controlling This Explicitly

- `std::atomic` operations take an optional `memory_order` argument:
 - `memory_order_relaxed` — atomicity only, no ordering guarantee at all
 - `memory_order_acquire` — no later read/write in this thread can be reordered before this load
 - `memory_order_release` — no earlier read/write in this thread can be reordered after this store
 - `memory_order_acq_rel` / `memory_order_seq_cst` — combine both / add a single global total order
- Default (if you omit the argument) is `memory_order_seq_cst` — safest, but the most expensive

Acquire-Release in Practice: Publishing a Node

```
Node* n = new Node(val);
n->next = head.load(std::memory_order_relaxed);
while (!head.compare_exchange_weak(
    n->next, n, std::memory_order_release,
    std::memory_order_relaxed));

// a reader that later does head.load(memory_order_acquire)
// is guaranteed to see n's fully-initialized contents, not a half-built node
```

Not All Hardware Reorders the Same Way

- x86 (TSO — Total Store Order): fairly strong. Only StoreLoad reordering is allowed — exactly what we just saw
- ARM / POWER (weak memory models): loads and stores can reorder much more freely — StoreStore, LoadLoad, and LoadStore too
- Consequence: lock-free code that “works” with relaxed ordering on your x86 laptop can break silently on an ARM server or phone
- This is exactly why we write `memory_order` explicitly instead of relying on “what this CPU happens to do”

The Problem RCU Doesn't Fully Solve

- RCU is excellent for read-mostly, kernel-style data (routing tables, config) with clear grace periods
- For general-purpose lock-free structures (stacks, queues, maps) under heavy read+write churn, two lighter-weight schemes are more common in practice:
 - Hazard pointers
 - Epoch-based reclamation
- Both answer the same question: “I want to free this node — how do I know no other thread still holds a pointer to it?”

Hazard Pointers

- Each thread publishes, in a shared per-thread slot, the pointer(s) it is currently dereferencing
 - Before reclaiming a retired node, a thread scans all hazard-pointer slots
 - If no slot holds that address, it's safe to free; otherwise, defer and retry later
- Bounded memory overhead (a small fixed table), no dependency on a global grace period
- Used in: Meta's folly library, Microsoft's C++ STL internals, Java's later concurrent collection implementations

Epoch-Based Reclamation

- Global epoch counter; each thread records the epoch it was in when it entered a lock-free operation
 - A retired node from epoch E is only actually freed once every thread has advanced past epoch E
 - Cheaper per-access than hazard pointers (no per-pointer publish), but garbage can pile up if one thread stalls mid-epoch
- Used in: crossbeam-epoch (Rust's standard lock-free toolkit), and internally by parts of the Linux RCU implementation

Lock-Free Ring Buffers: The SPSC Queue

- Single-Producer/Single-Consumer circular buffer — the workhorse of low-latency pipelines
- Producer only ever writes the head index; consumer only ever writes the tail index
 - Because each index has exactly one writer, no CAS is needed at all — just plain atomic loads/stores with acquire/release
- This is how NIC packet rings, and most market-data ingestion pipelines in HFT systems, move data between a network thread and a processing thread

The LMAX Disruptor Pattern

- Generalizes the ring buffer to multiple producers/consumers without locks
- Pre-allocated fixed-size ring; producers claim slots via an atomic published sequence number instead of a lock
 - Each consumer tracks its own read cursor and can process independently — batching amortizes cache-coherence traffic across many events
- Origin: LMAX's exchange matching engine — published throughput over 6 million orders/sec on a single thread pinned to a core
- A good closing example: this is per-core sharding + a lock-free SPSC/MPMC ring buffer + memory ordering, all the ideas from this course combined

Problems with Lock-Free Programming

- Usually, it is hard to write lock-free code
- It is even harder to write correct lock-free code
- Spin-Locks causes heavy memory usage
- Overall-performance can go down in some cases compared to mutexes
- ABA Problem may occur in some cases

How to decide between lock-based and lock-free?

Lock-Based vs Lock-Free

- Observe the number of threads:
 - If number of threads are very high then lock-free may be better
- Observe the contention period:
 - If low then opt for lock-free otherwise, use lock-based (why?)

Rule of Thumb: Test your code and measure the performance

References

- [Is Parallel Programming Hard, And, If So, What Can You Do About It?](#) By Paul Mckenney
- [C++ Concurrency in Action](#) By Anthony Williams
- [std::atomic - cppreference](#)
- More about RCU - [Link](#)
- An awesome step-by-step guide to Lock-Free Programming - [Link](#)